



MANIPAL

ACADEMY *of* HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

Linux OS and Scripting – 4_Theory

Reg. Number	Name	Branch
261100690006	MOHAMMED HARIS	Cybersecurity

10/09/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES

MANIPAL

(A constituent unit of MAHE, Manipal)

Hard Link vs. Soft Link

Part A — Create the links

Create a directory named `link_lab` and navigate to it.

Create a file named `security.txt` containing the following text:

Linux File System Security Lab

Confidential Security Report

Create:

a hard link named `security_hard.txt` a

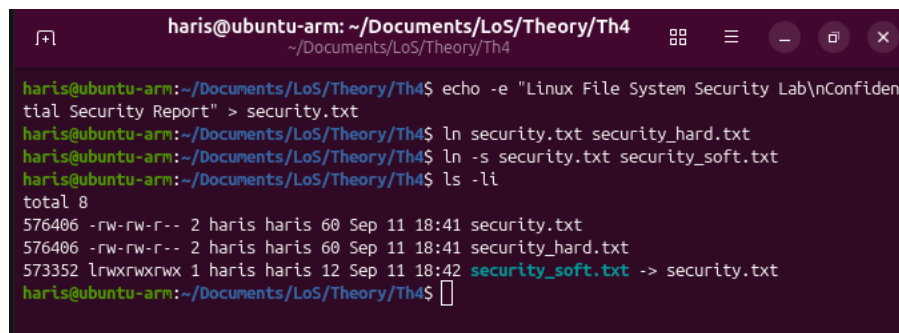
symbolic link named `security_soft.txt`

List all three files using a command that displays their inode numbers.

Record the inode numbers of:

`security.txt` `security_hard.txt`

`security_soft.txt`



```

haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ echo -e "Linux File System Security Lab\nConfiden
tial Security Report" > security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ ln security.txt security_hard.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ ln -s security.txt security_soft.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ ls -li
total 8
576406 -rw-rw-r-- 2 haris haris 60 Sep 11 18:41 security.txt
576406 -rw-rw-r-- 2 haris haris 60 Sep 11 18:41 security_hard.txt
573352 lrwxrwxrwx 1 haris haris 12 Sep 11 18:42 security_soft.txt -> security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$

```

What difference do you observe?

`security.txt` and `security_hard.txt` share the same inode number, indicating that both names refer to the same file.

`security_soft.txt` has a different inode number because it is a separate file that simply stores the path to `security.txt`.

Part B — Examine File Metadata

Use `ls -l` to examine the three files. Identify the file type of each file from the first character of the permission field.

Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.

```

haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ stat security_hard.txt
  File: security_hard.txt
  size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 259,2    Inode: 576406      Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   haris)   Gid: ( 1000/   haris)
Access: 2026-09-11 18:41:52.903289563 +0530
Modify: 2026-09-11 18:41:52.659115809 +0530
Change: 2026-09-11 18:42:13.865598179 +0530
 Birth: 2026-09-11 18:41:52.658115097 +0530
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ stat security_soft.txt
  File: 'security_soft.txt' -> 'security.txt'
  size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 259,2    Inode: 573352      Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/   haris)   Gid: ( 1000/   haris)
Access: 2026-09-11 18:42:32.556422351 +0530
Modify: 2026-09-11 18:42:32.555370352 +0530
Change: 2026-09-11 18:42:32.555370352 +0530
 Birth: 2026-09-11 18:42:32.555370352 +0530
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$

```

Use the stat command on all three files and record the following information:

Parameter	Original	Hard Link	Soft Link
Inode number	576406	576406	573352
File type	Regular file	Regular file	Symbolic link
Size	60 bytes	60 bytes	12 bytes
Link count	2	2	1
Permissions	0664	0664	0664

Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

The original file and the hard link are both 60 bytes because they point to the same file. The soft link is only 12 bytes because it stores the name/path of the original file, not its actual contents.

File type — first character of permission field

File	First char	Type
security.txt	-	Regular file
security_hard.txt	-	Regular file
security_soft.txt	l	Symbolic link

Part C — Investigate readlink

1. Execute readlink on security_soft.txt. What information is displayed?

readlink security_soft.txt

The output is: security.txt

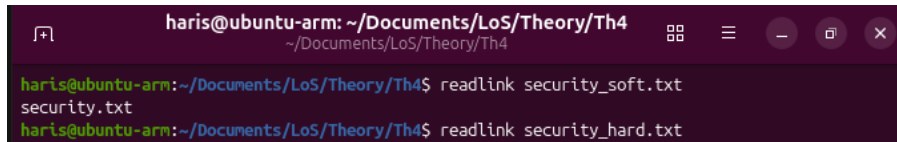
This shows that security_soft.txt is a symbolic link pointing to security.txt.

2. Execute readlink on security_hard.txt. What do you observe?

```
readlink security_hard.txt
```

Observation:

There is no output because security_hard.txt is a hard link. A hard link does not store the pathname of the original file; it directly refers to the same inode.



```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink security_soft.txt
security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink security_hard.txt
```

3. Based on your observations, explain why readlink works differently for hard links and symbolic links.

Use: `readlink -f security_soft.txt`

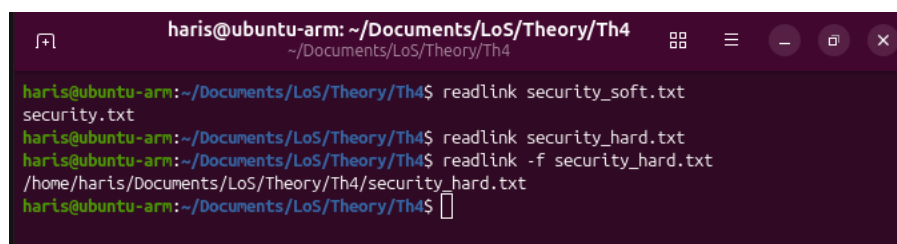
It displays the complete absolute path of the final target of the symbolic link, for example:

```
/home/aduuu/link_lab/security.txt
```

Explanation: readlink works differently because a symbolic link stores the pathname of its target, so readlink can display that pathname. A hard link does not store a target pathname; it directly refers to the same inode as the original file, so readlink security_hard.txt produces no output. readlink -f follows the symbolic link and resolves it to the final absolute path.

4. What does the -f option reveal?

The -f option reveals the complete absolute path of the final target of the symbolic link.



```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink security_soft.txt
security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink security_hard.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink -f security_hard.txt
/home/haris/Documents/LoS/Theory/Th4/security_hard.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

Part D — Content Modification

1. Append the following line to security.txt: Unauthorized login detected

This appends the line to security.txt without deleting the existing content.

2. Display the contents using all three filenames.

```
cat security.txt cat
```

```
security_hard.txt
```

```
cat security_soft.txt
```

The line Unauthorized login detected appears in all three outputs, because the hard link and symbolic link both access the same file content.

3. Is the newly added content visible through both links? Explain why

Yes. The newly added content is visible through both links.

- security_hard.txt is a hard link to the same inode and file data as security.txt.
- security_soft.txt is a symbolic link that points to security.txt.

Therefore, when content is added to security.txt, the updated content is accessible through both links.

```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink security_soft.txt
security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink security_hard.txt
security_hard.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink -f security_hard.txt
/home/haris/Documents/LoS/Theory/Th4/security_hard.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ echo "Unauthorised login detected" >> security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorised login detected
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorised login detected
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorised login detected
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ echo "System security alert" >> security_hard.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorised login detected
System security alert
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ ls
security.txt security_hard.txt security_soft.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

Part E — Rename Experiment

security.txt to:

incident.txt

1. Try accessing the content through security_hard.txt. Does it still work?

```
mv security.txt incident.txt
```

```
cat security_hard.txt
```

WORKS

2. Try accessing the content through security_soft.txt. Does it still work?

```
cat security_soft.txt  
DOESN'T WORK
```

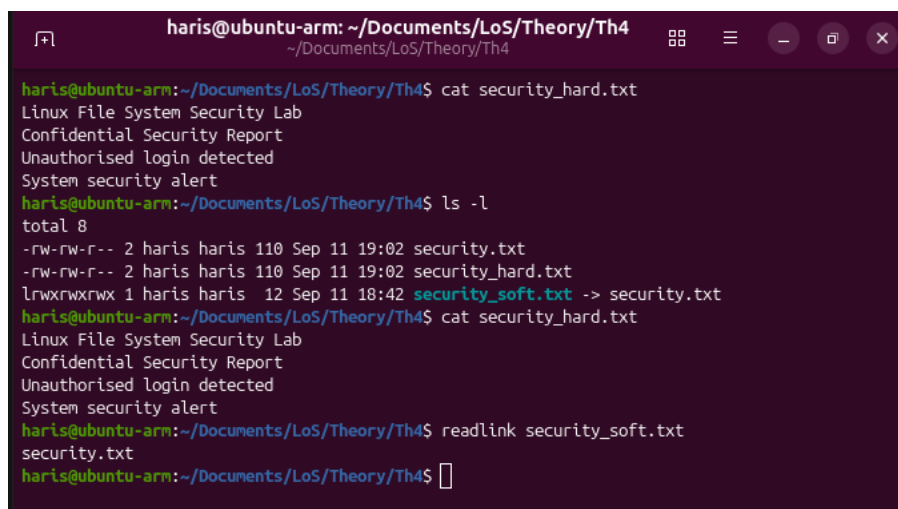
3. Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?

It will display: **security.txt**

This is because the symbolic link stores the original pathname, even though the file was renamed to incident.txt.

4. Explain why renaming the original file affects the symbolic link differently from the hard link.

- **Hard link:** security_hard.txt still works and displays the file's contents. Hard links point directly to the inode, so renaming security.txt does not affect them.
- **Soft link:** If security_soft.txt points to security.txt, it stops working because the original pathname no longer exists. The link becomes broken.



```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4  
~/Documents/LoS/Theory/Th4  
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat security_hard.txt  
Linux File System Security Lab  
Confidential Security Report  
Unauthorised login detected  
System security alert  
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ ls -l  
total 8  
-rw-rw-r-- 2 haris haris 110 Sep 11 19:02 security.txt  
-rw-rw-r-- 2 haris haris 110 Sep 11 19:02 security_hard.txt  
lrwxrwxrwx 1 haris haris 12 Sep 11 18:42 security_soft.txt -> security.txt  
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat security_hard.txt  
Linux File System Security Lab  
Confidential Security Report  
Unauthorised login detected  
System security alert  
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink security_soft.txt  
security.txt  
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

Part F — Deletion Experiment

1. Recreate the links if necessary and then perform the following.

```
security_hard.txt
```

2. Delete the original file.

```
rm security.txt
```

3. Check whether security_hard.txt can still be accessed.

```
cat security_hard.txt
```

Yes, security_hard.txt can still be accessed and will display the file contents. This is because the hard link refers directly to the same inode, even after the original filename is deleted.

4. Check whether security_soft.txt can still be accessed.

```
cat security_soft.txt
```

No, security_soft.txt cannot be accessed. It will give a “No such file or directory” error because its target security.txt has been deleted. The symbolic link becomes a dangling/broken link.

5. Use ls -l and readlink to investigate the symbolic link after deleting its target.

```
ls -l security_soft.txt
```

```
readlink security_soft.txt
```

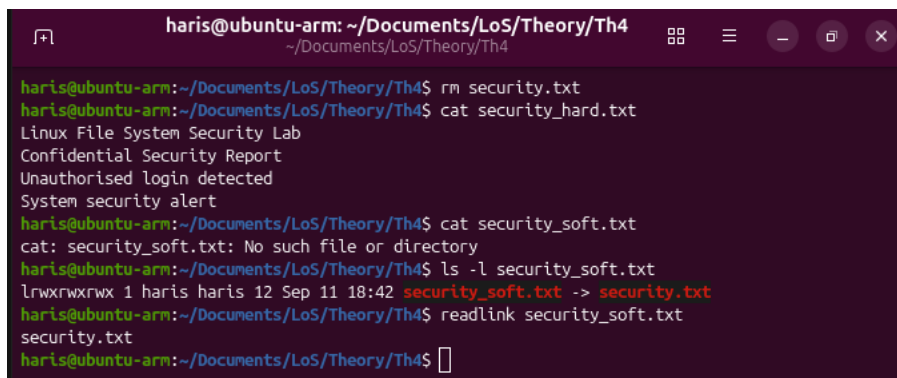
ls -l shows:

```
security_soft.txt -> security.txt
```

This shows the symbolic link still exists, but its target has been deleted.

readlink displays: security.txt

This confirms that the link still stores the old target path. security_soft.txt is now a dangling/broken symbolic link.



```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ rm security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorised login detected
System security alert
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ ls -l security_soft.txt
lrwxrwxrwx 1 haris haris 12 Sep 11 18:42 security_soft.txt -> security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ readlink security_soft.txt
security.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

6. What is a dangling/broken symbolic link?

A dangling/broken symbolic link is a symbolic link whose target file or path no longer exists.

7. Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode.

No. Deleting the original filename does not immediately delete the underlying data when a hard link exists.

Before deletion, the inode has a link count of 2 (security.txt and security_hard.txt).

After deleting security.txt, the link count becomes 1.

The same inode and underlying data remain, so security_hard.txt can still access the contents.

The data is removed only when the link count reaches 0 (and there are no open references).

Part G — Directory and Filesystem Behaviour

1. Create a directory named evidence. Attempt to create a hard link to this directory. Record the result. `mkdir evidence ln evidence evidence_hard`

Expected result: `ln: evidence: hard link not allowed for directory`

The hard link creation fails because hard links to directories are not normally allowed.

2. Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful? Yes, the operation is successful.

`ln -s evidence evidence_link`

The symbolic link `evidence_link` is successfully created and points to the evidence directory. Symbolic links can point to directories.

3. Based on your experiment, compare hard and symbolic links with respect to directory linking.

Hard link: Cannot normally be created for a directory. The attempt fails.

Symbolic link: Can be created for a directory successfully. `evidence_link` points to the evidence directory.

If multiple filesystems/mount points are available in the laboratory environment, attempt to create:

a hard link across filesystems

Hard links cannot span different filesystems because they reference the same inode. Therefore, attempting to create one across filesystems fails with an error such as ***Invalid cross-device link***.

Symbolic links, however, can point to files on another filesystem because they store the target's pathname:

Final Comparison

Parameter	Hard Link	Symbolic Link
Inode number	Same as target	Different from target
File type	Regular file	Symbolic link
Link count	Increases	Usually 1
File size	Same as target	Size of stored pathname
Stores target pathname	No	Yes
readlink output	No output	Shows target pathname
Effect of modifying target	Change is visible	Change is visible
Effect of renaming target	Still works	Becomes broken
Effect of deleting target	Still works if another hard link exists	Becomes broken
Can become a dangling or broken link?	No	Yes
Directory linking	Not normally allowed	Allowed
Cross-filesystem linking	Not allowed	Allowed

sort

Suppose alerts.txt contains:

192.168.1.10 admin 5

10.0.0.8 root 12

172.16.0.5 guest 2

10.0.0.20 admin 8

Format:

IP-address username failed attempts

Sort according to failed attempts

```

haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ nano alerts.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort -k3 -n alerts.txt
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$

```

Sort a list of suspicious IP addresses alphabetically.

```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ nano suspicious_ips.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort -u suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

Display only unique suspicious IP addresses using sort.

```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort -u suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

sort -u sorts the suspicious IP addresses and displays only unique IP addresses.

Sort a list of detected port numbers numerically.

```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ nano ports.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ cat ports.txt
22
80
443
8080
21
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort -n ports.txt
21
22
80
443
8080
```

sort -n sorts the detected port numbers in numerical order from smallest to largest.

Display port numbers in descending numerical order.

```
8080
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort -nr ports.txt
8080
443
80
22
21
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

Sort usernames without considering uppercase/lowercase differences.

```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ nano usernames.txt
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort -f usernames.txt
Alice
alice
BOB
bob
CHARLIE
charlie
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.

```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ nano failed_login.log
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort failed_login.log | uniq -c | sort -nr
  2 10.0.0.8
  1 192.168.1.10
  1 172.16.0.5
  1 10.0.0.20
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

Given IP,username,failed_attempts, sort records based on the number of failed attempts.

```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ nano records.csv
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ sort -t',' -k3 -n records.csv
172.16.0.5,guest,2
192.168.1.10,admin,5
10.0.0.20,admin,8
10.0.0.8,root,12
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

The records are sorted in ascending order based on the third field, which represents the number of failed attempts.

Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses.

```
haris@ubuntu-arm: ~/Documents/LoS/Theory/Th4
~/Documents/LoS/Theory/Th4
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' failed_login.log | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
haris@ubuntu-arm:~/Documents/LoS/Theory/Th4$
```

The command extracts IP addresses from the log, sorts them, and displays only unique IP addresses.

Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

```
harris@ubuntu-arn:~/Documents/LoS/Theory/Th4$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' failed_login.log | sort -u | uniq -c | sort -nr | head -5
1 192.168.1.10
1 172.16.0.5
1 10.0.0.8
1 10.0.0.20
harris@ubuntu-arn:~/Documents/LoS/Theory/Th4$
```

The pipeline extracts IP addresses, counts their occurrences, sorts them by frequency in descending order, and displays the top 5.